

Every `POST /api/v1/telehealth/intake/unified` payload field maps to one or more destinations on the server: a column on `encounters` / `patient_charts`, a tier-1/2/3 JSON storage column, an `encounter_intake_answers` row, or an upserted chart-side record (medication / allergy / condition / vital / address / document). This page documents that mapping so integrators can predict — without reading source code — exactly where their data lands.

Storage tiers (legend below). `tier 1` Strict — core demographics on patient/chart. Validation always required. `tier 2` Flexible — vitals + medical history. Validated but not strictly typed. `tier 3` Lax — encounter-specific Q&A. Provider-reviewable. `chart-sync` IntakeAnswerRouter mirrors this to chart-side rows. `document` Persists as a `patient_documents` row.

ON THIS PAGE

[Patient identity](#) [Addresses](#) [Vitals](#) [Medical history](#) [Encounter answers](#)
[Product / package selection](#) [Payment block](#) [Documents & identification](#) [Consents](#)
[Fulfillment preferences](#) [Sandbox & metadata](#)

Patient identity (tier 1)

Always required. `UnifiedIntakeController` uses these to find an existing `users` row (by `email`) or create a new one, plus a paired `patient_charts` row.

API field	Destination	Notes
<code>patient.first_name</code>	<code>users.first_name</code> + <code>patient_charts.first_name</code>	Used for test-name detection (matches <code>/\bttest(ing er)?\b/i</code> → flips <code>encounter.is_sandbox=true</code>).
<code>patient.last_name</code>	<code>users.last_name</code> + <code>patient_charts.last_name</code>	Same test-name detection.
<code>patient.email</code>	<code>users.email</code> (unique key)	If a user already exists with this email, the chart is reused; otherwise a new user is created.
<code>patient.date_of_birth</code>	<code>patient_charts.date_of_birth</code>	ISO date, must be before today.
<code>patient.phone</code>	<code>users.phone</code> + <code>patient_charts.phone</code>	Stored as-is; downstream notification services normalize to E.164.
<code>patient.gender</code>	<code>patient_charts.gender</code>	Accepts string (<code>male</code> / <code>female</code> / <code>other</code>) or numeric (<code>1</code> / <code>2</code> / <code>3</code>).
<code>patient.race</code>	<code>patient_charts.race</code>	Free-form string.
<code>patient.ethnicity</code>	<code>patient_charts.ethnicity</code>	Free-form string.

API field	Destination	Notes
patient.preferred_language	patient_charts.preferred_language	Free-form (e.g. <code>en</code> , <code>es</code>).

Addresses (tier 1)

One of three shapes; the controller normalizes to two `addresses` rows on the chart (one SHIPPING default, one BILLING default).

Shape	What it does
patient.address.{...}	Legacy single. Used as both shipping + billing default. State licensing is determined from this since "physical = shipping" on this platform.
patient.shipping_address.{...} + patient.billing_address.{...}	Distinct addresses. Shipping drives state licensing.
patient.shipping_address.{...} + patient.billing_same_as_shipping=true	Billing auto-copied from shipping at chart-create time.

Sub-fields on every address shape: `street` , `street2` , `city` , `state` (2-letter), `zip` , `country` (2-letter, default US).

Vitals (tier 2 + chart-sync)

Persisted to the `encounters.tier2_vitals` JSON snapshot AND to `patient_vitals` rows via IntakeAnswerRouter . tier 2 chart-sync

API field	Destination	Notes
vitals.height_inches	patient_vitals (type=height) + tier2_vitals.height_inches	Range 30–96. Used for BMI computation downstream.
vitals.weight_lbs	patient_vitals (type=weight) + tier2_vitals.weight_lbs	Range 50–800. Auto-flagged as the patient's "starting_weight" if no prior weight reading exists.
vitals.blood_pressure_systolic / .blood_pressure_diastolic	patient_vitals (type=blood_pressure) + tier2_vitals.blood_pressure_*	Both required together.
vitals.heart_rate_bpm	patient_vitals (type=heart_rate) + tier2_vitals.heart_rate_bpm	Range 40–200.

Medical history (tier 2 + chart-sync)

Three string arrays, each routed to chart-side rows via the typeahead-aware `IntakeAnswerRouter`. Strings are matched against `terminology_*` tables — unmatched strings are persisted as free-text on the chart row. tier 2 chart-sync

API field	Destination	Notes
<code>medical_history.allergies[]</code>	<code>patient_allergies</code> (one row per string) + <code>tier2_medical_history.allergies</code>	Empty / "None" / "NKDA" results in a row with <code>has_no_allergies=true</code> .
<code>medical_history.medications[]</code>	<code>patient_medications</code> (one row per string) + <code>tier2_medical_history.medications</code>	String-only — for structured dose/frequency, use the <code>/me/patient/medications</code> endpoint instead.
<code>medical_history.conditions[]</code>	<code>patient_conditions</code> (one row per string) + <code>tier2_medical_history.conditions</code>	Matches <code>terminology_conditions</code> by display name; unmatched → free-text row.

► How chart sync works (deeper dive)

Encounter answers (tier 3 + chart-sync)

Free-form key/value object keyed by field `slug`. Values are persisted in `tier3_encounter_questions` JSON AND as individual `encounter_intake_answers` rows (one per slug). For slugs that map to standard chart fields (e.g. `current_weight_lbs`) the answer also flows through `IntakeAnswerRouter` to the chart. tier 3 chart-sync

Discover slugs: `GET /api/v1/telehealth/encounter-types/{id}/schema` returns the canonical list of fields + types + display conditions for an encounter type.

Field type	Expected value	Example
text / textarea	string	<code>"chief_complaint": "Mild headache"</code>
number	number	<code>"current_weight_lbs": 215</code>
yes_no	"Yes" or "No" (case-insensitive)	<code>"glp1_diabetes_mellitus": "No"</code>
select	option value string	<code>"weight_goal": "lose_15_lbs"</code>
multiselect	array of option values	<code>"prior_attempts": ["diet", "exercise"]</code>
toggle / boolean	true / false	<code>"sms_opt_in": true</code>
height (composite)	<code>{"feet":N,"inches":N}</code>	<code>"height": {"feet": 5, "inches": 10}</code>
blood_pressure (composite)	<code>{"systolic":N,"diastolic":N}</code>	<code>"bp": {"systolic": 120, "diastolic": 80}</code>

Field type	Expected value	Example
date	ISO date (YYYY-MM-DD)	"last_period": "2026-04-15"

`ApiIntakeDependencyValidator` runs after the basic schema validation and enforces conditional **required** rules (e.g., a follow-up answer required when a parent answer is "Yes"). Failures return 422 with field-level error messages.

Product / package selection

Two valid shapes for catalog selection — pick one per integration.

Modern nested shape (preferred)

API field	Resolves to	Notes
<code>products[].product_id</code>	<code>products.id</code>	UUID — exact SKU.
<code>products[].product_number</code>	<code>products.product_number</code>	Human-friendly <code>PROD-XXXXX</code> .
<code>products[].product_type_id</code>	<code>product_types.id</code> → resolved placeholder <code>product</code>	Generic — provider picks the final SKU at prescribe-time.
<code>products[].product_type_slug</code>	<code>product_types.slug</code> → placeholder	e.g. <code>"tirzepatide"</code> .
<code>products[].dose_product_id</code>	<code>dose_products.id</code>	Per-line dose pin (rare — usually provider-decided).
<code>products[].quantity</code>	<code>encounter_products.quantity</code>	1–99.
<code>products[].snapshot_price</code>	<code>encounter_products.snapshot_price</code>	Locks the patient-charge price.
<code>packages[].package_id</code>	<code>packages.id</code>	UUID.
<code>packages[].package_number</code>	<code>packages.package_number</code>	<code>PKG-XXXXX</code> .
<code>packages[].plan_id</code>	<code>package_plans.id</code>	Subscription term selection.

Legacy flat shape (backward compat)

`product_ids[]` , `package_id` , `package_ids[]` , `plan_id` — accepted but new integrations should use `products[]` / `packages[]` .

Payment block

Five modes — pick exactly one. Sales-org and client tokens cannot use `authorize` (PRX is not the merchant of record on API flows).

payment.mode	Required fields	Order outcome
authorize	gateway, amount, card_token	PRX runs pre-auth. Order workflow → <code>READY_FOR_CAPTURE</code> . Embed-only.
reference_preauth	pre_auth.transaction_id, pre_auth.authorized_amount	You ran a pre-auth; we record + vault. Capture later via <code>POST /orders/{id}/capture</code> .
reference_captured	transaction.transaction_id, transaction.amount	Record-only audit trail of a sale you already ran.
prepaid	reason	No charge — out-of-band billing settled. Order workflow → <code>READY_FOR_FULFILLMENT</code> .
patient_review	– (no extra fields)	Defer — patient gets emailed a portal link to confirm + pay.

`payment.gateway` in `authorize` mode must be one of `authorize_net` / `stripe` / `nmi` / `square` . In record-only modes, it accepts any free-form string (the integrator's gateway name, recorded for audit).

Documents & identification

`identification.{...}` stores structured ID metadata (number + state + type) on the patient chart. `documents[]` uploads up to 20 files (10MB each), persisted as `patient_documents` rows linked to both the chart and the encounter. [document](#)

API field	Destination	Notes
<code>identification.id_number</code>	<code>patient_charts.government_id_number</code>	Required on controlled-substance encounter types.
<code>identification.id_state</code>	<code>patient_charts.government_id_state</code>	2-letter US state.
<code>identification.id_type</code>	<code>patient_charts.government_id_type</code>	<code>drivers_license</code> <code>state_id</code> <code>passport</code> <code>military_id</code> .
<code>documents[].type</code>	<code>patient_documents.document_type</code>	12 enum values — see OpenAPI.
<code>documents[].filename</code>	<code>patient_documents.original_filename</code>	Including extension.
<code>documents[].file_base64</code>	S3 (documents disk) → <code>patient_documents.path</code>	Preferred. MIME sniffed server-side; mismatch → 422.
<code>documents[].url</code>	Server fetches → S3 → <code>patient_documents.path</code>	Use only when <code>file_base64</code> isn't practical (large files). HTTPS only.

Consents

Each entry creates a `consent_audit_logs` row + (when applicable) a `patient_chart_consent` upsert.

API field	Destination	Notes
<code>consents[].consent_type</code>	<code>consent_audit_logs.consent_type</code>	Integer enum — see OpenAPI <code>ConsentType</code> .
<code>consents[].consent_text</code>	<code>consent_audit_logs.consent_text</code>	Full text shown to user (kept for audit).
<code>consents[].consent_version</code>	<code>consent_audit_logs.consent_version</code>	Free-form (e.g. "2.1").
<code>consents[].consented_at</code>	<code>consent_audit_logs.consented_at</code>	If omitted, server stamps <code>now()</code> .
<code>consents[].ip_address</code>	<code>consent_audit_logs.ip_address</code>	If omitted, server captures <code>\$request->ip()</code> .
<code>consents[].signature_method</code>	<code>consent_audit_logs.signature_method</code>	<code>click</code> <code>typed</code> <code>pad</code> .
<code>consents[].signature_data</code>	<code>consent_audit_logs.signature_data</code>	Base64 PNG (for <code>pad</code> method).

Fulfillment preferences

All optional. Resolution chain documented in `docs/dev/FC_OVERRIDES_ARCHITECTURE.md §3` — null fields inherit FC defaults / vendor defaults.

API field	Destination	Notes
<code>fulfillment_preferences.shipping_service_code</code>	<code>encounters.fulfillment_preferences.shipping_service_code</code>	Per-FC service code (e.g. <code>USPS_GROUND</code>).
<code>fulfillment_preferences.bill_to</code>	<code>encounters.fulfillment_preferences.bill_to</code>	<code>Patient</code> <code>Practice</code> <code>Customer</code> .
<code>fulfillment_preferences.ship_to</code>	<code>encounters.fulfillment_preferences.ship_to</code>	<code>Patient</code> <code>Practice</code> .
<code>fulfillment_preferences.per_product[]</code>	<code>encounter_products.fulfillment_overrides</code> JSON	Per-line override — most specific wins.

Sandbox & metadata

API field	Destination	Notes
<code>is_sandbox</code>	<code>encounters.is_sandbox</code> + <code>orders.is_sandbox</code>	Skips provider auto-assignment, fulfillment dispatch, commission/billing reports. Auto-set when patient name matches <code>/\btest(ing er)?\b/i</code> .

API field	Destination	Notes
reason_for_visit	encounters.reason_for_visit	Free-form (max 500 chars).
chief_complaint	encounters.chief_complaint	Free-form (max 1000 chars).
metadata	encounters.metadata JSON	Opaque pass-through — admin UI surfaces it on the encounter detail page.
client_id / client_number	encounter.client_id (resolved to UUID)	Falls back to authenticated token's tenant.
sales_org_id / sales_org_number	encounter.sales_organization_id	Falls back to client's parent org.
provider_id	encounter.provider_profile_id	Optional — if omitted, <code>EncounterAssignmentEngine</code> picks one (unless <code>is_sandbox=true</code>).

Tip — confirming where your data landed. After a successful intake, `GET /api/v1/encounters/{id}?include=patient,prescriptions,soapNotes` returns the full encounter + chart-side rows that resulted from the upsert pipeline. Use this to verify medications/allergies/conditions were matched to terminology rows (vs. persisted as free-text), and to confirm the encounter status transitioned correctly.

Source of truth

This page is hand-curated from

`app/Http/Controllers/Api/V1/Telehealth/UnifiedIntakeController.php::store()` and `app/Services/Encounter/IntakeAnswerRouter.php` . When the validation rules or chart-sync routing change, this page must be updated as part of the same commit (see `feedback_api_docs_update_rule`).